

FASE 0 — Auditoría, inventario y plan de cambio

Proyecto: BuscoEdu — buscoedu-platform

Fecha: 2026-08-29

Estado: completada

1. Método y fuente de verdad

No fue posible conectarse en vivo a Supabase durante esta intervención porque el entorno de ejecución del agente **no dispone de las variables de conexión** (`NEXT_PUBLIC_SUPABASE_URL` , `NEXT_PUBLIC_SUPABASE_ANON_KEY` , `SUPABASE_SERVICE_ROLE_KEY` no están presentes). Ante esa limitación, la fuente de verdad utilizada — por orden de prioridad — fue:

1. `BUSCOEDU_DATA_DICTIONARY (1).md` — snapshot técnico extraído directamente de `information_schema` el 15-ago-2026. Es el documento que describe **lo que existe hoy** (57 tablas, 145 FKs, 60 CHECK). **Fuente principal.**
2. **Migraciones del repositorio** (`supabase/migrations/*.sql` y `EJECUTAR_EN_SUPABASE.sql`) — DDL real ejecutado para el módulo admin.
3. `BUSCOEDU_DATABASE_SCHEMA_FINAL.md` — diseño conceptual (el “por qué”).
4. **Código que consulta/escribe datos** (`src/lib/*` , `app/**`).

⚠ **Puerta de control:** todas las migraciones de la Fase 1 se escriben de forma **idempotente y aditiva** (`IF NOT EXISTS` , `ADD COLUMN IF NOT EXISTS` , `CREATE POLICY` con `DROP POLICY IF EXISTS` previo). Deben ejecutarse en Supabase por una persona con acceso; antes de aplicarlas en producción **se debe correr el bloque de verificación de columnas incluido en cada migración** para confirmar que el nombre real coincide con el documentado. Esto neutraliza el riesgo de operar sin introspección en vivo.

2. Inventario del repositorio

- **Stack detectado:** Next.js `latest` (App Router), React `latest` , TypeScript, Tailwind 3.4, `@supabase/ssr 0.12` y `@supabase/supabase-js 2.112`.
- **Cliente Supabase:** `src/lib/supabase.ts` → `getSupabaseClient()` (browser client, anon key, respeta RLS con la sesión del usuario). Patrón obligatorio.
- **Middleware:** `middleware.ts` protege `/admin/*` con `createServerClient` de `@supabase/ssr` , resuelve `auth.uid()` → `usuarios_internos.auth_user_id` → rol vía `roles.codigo` . Actualmente exige `super_admin` .
- **Módulos en producción (NO tocar):** `app/page.tsx` (landing), `app/explorar/` (explorador), `app/api/naia/route.ts` + `src/lib/naia-real.ts` (NaIA con Abacus.AI), `app/admin/**` (panel admin).
- **Estado Git:** `main` limpio, sin cambios sin commit. Última rama de trabajo activa.

- **Proceso de migraciones:** carpeta `supabase/migrations/` con convención `YYYYMMDDHHMMSS_descripcion.sql`. No hay CLI de rollback automatizado; el rollback es manual/documentado. Las migraciones se aplican vía SQL Editor de Supabase.

3. Tablas existentes relevantes al Lead Center (confirmadas en el diccionario)

Todas las tablas que el prompt exige reutilizar **existen** en el esquema real:

```
visitantes , personas , preferencias_educativas_persona ,
perfil_progresivo_persona , tipos_consentimiento , consentimientos_persona ,
etapas_embudo , subestados_oportunidad , oportunidades ,
historial_etapas_oportunidad , historial_scoring_oportunidad ,
reglas_estancamiento , asignaciones_oportunidad , tareas_crm , notas_crm ,
conversaciones , mensajes_conversacion , hechos_extraidos_naia , aplicaciones ,
propuestas_comerciales , versiones_propuesta_comercial ,
transferencias_universidad , comunicaciones_transaccionales , auditoria_eventos ,
eventos_negocio , roles , usuarios_internos , usuarios_universidad ,
ofertas_academicas , programas_academicos , sedes , universidades .
```

Conclusión: el modelo de datos del CRM ya está creado. El Lead Center es mayoritariamente **capa de lectura/escritura sobre lo existente**; las extensiones son mínimas.

4. Matriz Necesidad → Tabla/campo existente → Reutilización → Extensión mínima

Necesidad	Tabla/campo existente	Reutilización	Extensión mínima necesaria
Identidad de la persona	personas.id (uuid PK)	Total	Ninguna
Llave funcional por celular E.164	personas.telefono_principal (texto sin normalizar)	Parcial	+ personas.celular_e164 (texto) + índice único parcial + personas.pais_celular
Vínculo persona ↔ Supabase Auth	(no existe en personas)	—	+ personas.auth_user_id uuid → auth.users (único parcial)
Estado de verificación de teléfono/ WhatsApp	(no existe)	—	+ personas.telefono_verificado , personas.whatsapp_verificado , personas.metodo_verificacion
Vincular visitante con persona	personas.visitante_id → visitantes	Total	Ninguna
Reto/desafío OTP con hash, caducidad, intentos	(no existe ninguna tabla OTP)	—	+ tabla nueva desafios_otp (capacidad inexistente, no duplica ningún concepto)
Modelo de negocio de la oferta	(no existe en ofertas_academicas)	—	+ ofertas_academicas.modelo_negocio CHECK ('por_lead','por_inscrito')
Snapshot del modelo de negocio en la oportunidad	oportunidades (sin campo modelo)	—	+ oportunidades.modelo_negocio_snapshot
Clave de idempotencia de la conversión	(no existe)	—	+ oportunidades.clave_idempoten

Necesidad	Tabla/campo existente	Reutilización	Extensión mínima necesaria
			cia (texto, único parcial)
Consentimiento general vs específico	tipos_consentimiento + consentimientos_persona (con autoriza_contacto/whatsapp/transferencia, version_texto, ip_origen, universidad_id, estado)	Total	Ninguna (seed de tipos si faltan)
Aplicación	aplicaciones	Total	Ninguna
Oportunidad en embudo	oportunidades + etapas_embudo + subestados_oportunidad	Total	Ninguna
Propuesta + versión inmutable	propuestas_comerciales + versiones_propuesta_comercial (*_snapshot jsonb)	Total	Ninguna
Historial de etapas	historial_etapas_oportunidad	Total	Ninguna
Scoring auditable	historial_scoring_oportunidad	Total	Ninguna
Asignación (responsable vigente + historial)	oportunidades.asesor_asignado_id (vigente) + asignaciones_oportunidad (historial)	Total	Ninguna
Transferencia con consentimiento	transferencias_universidad	Total	Ninguna

Necesidad	Tabla/campo existente	Reutilización	Extensión mínima necesaria
	(consentimiento_id NOT NULL)		
Tareas / notas	tareas_crm , notas_crm	Total	Ninguna
Conversaciones / mensajes / hechos NaIA	conversaciones , mensajes_conversacion , hechos_extraidos_nai a	Total (línea de tiempo compone desde aquí)	Ninguna
Registrar contacto del asesor	conversaciones + mensajes_conversacion (canal asesor)	Total	Ninguna
Eventos de negocio	eventos_negocio	Total	Ninguna
Reglas de estancamiento	reglas_estancamiento	Total	Ninguna
Comunicaciones sin envío real	comunicaciones_transaccionales (estado_envio='pendiente')	Total	Ninguna
Copiloto: registrar decisión del asesor	notas_crm + auditoria_eventos	Total	Ninguna
Rol de acceso al Lead Center	roles (super_admin , asesor) + usuarios_internos	Total	Ninguna (seed asegura rol asesor)

Resumen de extensiones (todas aditivas)

1. `personas` : + `celular_e164` , + `pais_celular` , + `auth_user_id` , + `telefono_verificado` , + `whatsapp_verificado` , + `metodo_verificacion` .
2. `ofertas_academicas` : + `modelo_negocio` .
3. `oportunidades` : + `modelo_negocio_snapshot` , + `clave_idempotencia` .
4. **Tabla nueva** `desafios_otp` (única estructura nueva; cubre una capacidad inexistente: retos OTP con hash, caducidad, intentos y rate limiting).

Ninguna extensión duplica un concepto existente. **No** se crean tablas paralelas a `personas` , `oportunidades` , `funnel` , `propuestas` , `historial` , `tareas` , `notas` o `transferencias`.

5. RLS, funciones e índices actuales

- Función `public.is_super_admin()` (SECURITY DEFINER) ya existe y resuelve el rol vía `usuarios_internos / roles`. Se **reutiliza** y se le suma una función hermana `public.es_asesor_o_super()` y helpers para el mapeo `auth.uid() → usuarios_internos.id`.
- RLS está habilitado en las tablas admin. Las tablas CRM del Lead Center requieren políticas específicas para `asesor` (solo su alcance) y `super_admin` (completo), que se crean en la Fase 1.
- Índices existentes cubren FKs del módulo admin. Se agregan índices para celular normalizado, etapas/subestados, asesor asignado, fechas de próxima acción y estados usados por dashboard/pipeline.

6. Riesgos detectados y mitigación

Riesgo	Mitigación
No hay introspección en vivo del esquema	Migraciones idempotentes + bloque de verificación de columnas antes de aplicar
Nombres de columna divergentes (p. ej. <code>pais_documento</code> vs <code>pais_documento_id</code>)	Migraciones solo agregan ; nunca renombran. Los <code>SELECT</code> del código usan solo columnas confirmadas en el diccionario
Operaciones multi-tabla no atómicas desde el cliente	Se implementa RPC transaccional <code>fn_convertir_aplicacion</code> en Postgres (una sola transacción)
Exposición de service role	Solo se usa en rutas de servidor (<code>app/api/**</code>), nunca con prefijo <code>NEXT_PUBLIC_</code>

7. Decisión para continuar

Continuar a Fase 1. El esquema está confirmado documentalmente, las extensiones son mínimas y aditivas, y no se detecta duplicación estructural ni riesgo de pérdida de datos. La única salvedad — ausencia de conexión en vivo — queda mitigada por el diseño idempotente y verificable de las migraciones.